2026 VLSI Design and Implementation

LAB1 Matrix Multiplication

Objection

In this lab, students are required to implement a hardware design for an **8x8 matrix multiplication** and learn RTL simulation and synthesis.

Lab Download

---

1. Download the provided environment using the following command.
   tar xvf /home/TA00/VLSI_2026/Lab1.tar
2. The extracted LAB directory contains:
   - `00_TESTBED`: Contains the testbench and pattern files .
   - `01_RTL`: Your workspace for the Verilog design (`MATRIX_MULT.v`).
   - `02_SYN`: Synthesis scripts and working directory.
   - `03_GATE`: Gate-level netlist and simulation files.

Design Description

The MATRIX_MULT module performs the multiplication of two 8x8 matrices, Matrix A and Matrix B. Both matrices contain 8-bit unsigned integers.

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

Fig. 1. 3x3 Matrix Multiplication Example

1. **Input Phase (16 Cycles)**: When the `in_valid` signal is high, the design receives one row of a matrix per clock cycle. Because each row **contains eight 8-bit integers**, the input port (`in_data`) is 64 bits wide. The testbench will transmit Matrix A completely, followed immediately by Matrix B. **Therefore, the continuous input phase lasts for exactly 16 cycles (Cycles 1 to 8 for Matrix A's rows, and Cycles 9 to 16 for Matrix B's rows).**

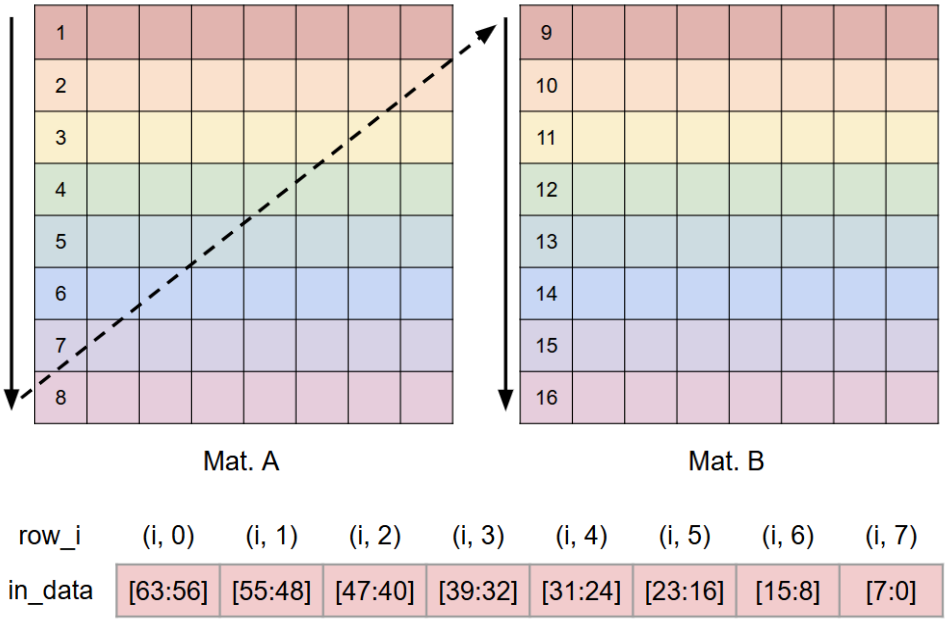

Fig. 2. Input order in 16 consecutive cycles

2. **Calculation Phase**: The core operation is standard matrix multiplication. An element in the output Matrix C at row 'i' and column 'j' is computed by multiplying the elements of the i-th row of Matrix A with the corresponding elements of the j-th column of Matrix B, and summing the 8 resulting products. Since multiplying two 8-bit numbers results in a 16-bit product, and the operation sums 8 of these products, the final value requires additional bit-width to prevent overflow. To ensure precision, each **output element is allocated 20 bits**.

3. **Output Phase (64 Cycles)**: Once the computation is ready, the module must assert the `out_valid` signal and begin outputting the resulting 8x8 Matrix C. The output is provided one element at a time (20 bits per cycle) through the `out_data` port. **The output sequence must strictly follow row-major order: starting from Row 0 (elements 0 to 7), followed by Row 1 (elements 0 to 7), and continuing this pattern until the final element of Row 7 is processed. This continuous output phase lasts for exactly 64 cycles.**

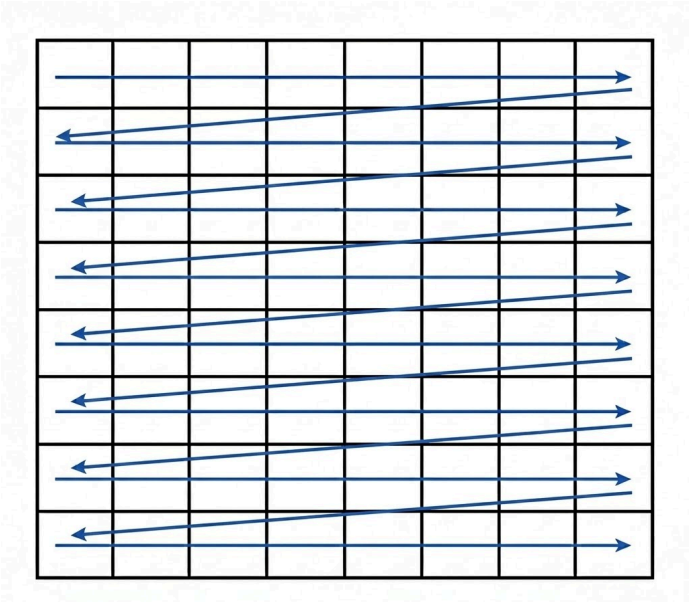

Fig. 3. Output order in 64 consecutive cycles

Signal Description

Signal	In / Out	Bit Width	Description
`clk`	In	1	System Clock.
`rst_n`	In	1	Asynchronous active-low reset.
`in_valid`	In	1	High when the input `in_data` is valid.
`in_data`	In	64	Contains one row of the input matrices. The data is formatted from MSB to LSB (e.g., `in_data[63:56]` is the 0th element of the row).
`out_valid`	Out	1	High when the output `out_data` is valid.
`out_data`	Out	20	A single 20-bit element of the calculated output matrix.

Table. 1. I/O port list

Design Constraints

01_RTL

1. Top module name: `MATRIX_MULT.v`
2. The reset signal (`rst_n`) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.
3. The `out_data` should be zero after your `out_valid` is pulled down.
4. The `out_valid` signal cannot overlap with the `in_valid` signal at any time.
5. The `out_valid` signal should be high in **64 consecutive cycles**.
6. The next inputs will come in the next 3 ~ 7 negative edges of the clock after your `out_valid` is pulled down.
7. **Your design must pass all hidden testcase.**

02_SYN

1. The synthesis result cannot contain any **latch**.
2. The input / output delay is set to 0.5 * (clock period), and the output loading is set to 0.05.
3. You can adjust your clock period by yourself. The precision of the clock period is 0.1, for example, 45.0 is allowed, 44.55 is not allowed.
4. After synthesis, the design must meet timing requirements. The area report is valid only if the final slack reported in `MATRIX_MULT.timing` is **MET**.

```
Startpoint: in_valid (input port clocked by clk)
Endpoint: mat_a_reg_1__0__0_
          (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

Point                                     Incr       Path
-----------------------------------------------------------
clock clk (rise edge)                     0.00       0.00
clock network delay (ideal)               0.00       0.00
input external delay                     10.00      10.00 f
in_valid (in)                             0.00      10.00 f
U1789/Y (INVX1)                           0.26      10.26 r
U1795/Y (NOR2XL)                          0.12      10.38 f
U3208/Y (NAND3BX1)                        0.15      10.53 r
U1791/Y (INVXL)                           0.11      10.64 f
U3339/Y (NAND3XL)                         0.14      10.78 r
U3602/Y (OR2X1)                           0.80      11.58 r
U1799/Y (CLKINVX3)                        0.62      12.20 f
U3659/Y (AO22X1)                          0.38      12.59 f
mat_a_reg_1__0__0_/D (DFFQX1)             0.00      12.59 f
data arrival time                                   12.59

clock clk (rise edge)                    20.00      20.00
clock network delay (ideal)               0.00      20.00
clock uncertainty                        -0.10      19.90
mat_a_reg_1__0__0_/CK (DFFQX1)            0.00      19.90 r
library setup time                       -0.23      19.67
data required time                                  19.67
-----------------------------------------------------------
data required time                                  19.67
data arrival time                                  -12.59
-----------------------------------------------------------
slack (MET)                                         7.08
```

Fig. 4. Timing report with no timing violation

5. You can use `08_check` to check your synthesis result.

```
[chfan@localhost 02_SYN]$ ./08_check
============================
--> V Latch Checked!
--> V Width Mismatch Checked!
--> V No Error in syn.log!
--> V Timing (MET) Checked!
============================
--> V 02_SYN Success !!
============================
Cycle: 20.00
Area: 63126.305690
Gate count: 6326
Dynamic: Total Dynamic Power    = 923.2034 uW  (100%)
Leakage: Cell Leakage Power     =  53.2685 uW
```

```
[chfan@localhost 02_SYN]$ ./08_check
============================
--> X There is Latch in this design ---
--> V Width Mismatch Checked!
--> V No Error in syn.log!
--> V Timing (MET) Checked!
============================
--> X 02_SYN Fail !! Please check out syn.log file.
============================
Cycle: 20.00
Area: 63126.305690
Gate count: 6326
Dynamic: Total Dynamic Power    = 923.2034 uW  (100%)
Leakage: Cell Leakage Power     =  53.2685 uW
```

Fig. 5. `08_check` example

03_GATE

1. All requirements in 01_RTL must be met after including timing checks in Gate-Level simulation.

Wavefrom Examples

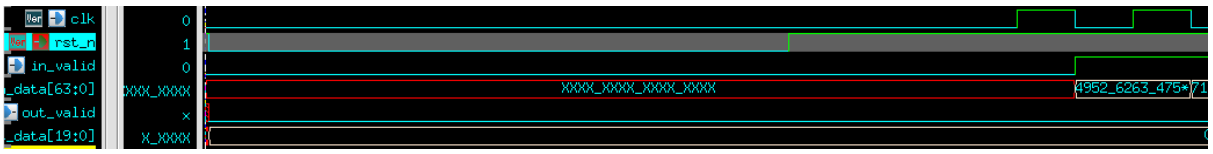

Fig. 6. The reset signal would be given only once at the beginning of simulation.

Fig. 7. input signal will be sent in 16 consecutive cycles.

Fig. 8. The next input will come in the next 3 ~ 7 negative edges of the clock.

Grading Policy

---

1. **Function Validity: 70%**
   Passing RTL and Gate-level simulation without timing violations.
2. **Performance: 30%**
   Performance Score = `Latency * Clock Cycle Time`
   ※The `Latency` is the clock cycles between the falling edge of the in_valid and the rising edge of the first out_valid.
   ※`Clock Cycle Time` is defined by yourself.
3. **Due date: 2026/4/10 12:00 p.m.**
   Please submit the following zip files to **E3 system** before 12:00 at noon on Apr. 10
   - vlsixx.zip (ex. vlsi00.zip)
     - vlsixx (folder)
       - MATRIX_MULT.v
       - {your cycle time}.txt (ex. 10.6.txt)

Notes

- Submission
  - 1st Demo
    Submit and pass all test cases before 12:00 noon on April 10, 2026. The score will be calculated according to the grading policy.

  - 2nd Demo
    If you do not pass all test cases or fail to submit (in the 1st Demo), and submit and pass all test cases before 12:00 noon on April 17, 2026, you will receive the Function score(70%).

- Reference Commands
  - 01_RTL/ (RTL simulation) ./01_run_vcs_rtl
  - 02_SYN/ (Synthesis) ./01_run_dc_shell
  - 03_GATE / (Gate-level simulation) ./01_run_vcs_gate

- How to change cycle time
  You can adjust cycle time in `00_TESTBED/PATTERN.v` and `02_SYN/syn.tcl`.
  ※**Note: Both cycle time configurations must be the same.**

00_TESTBED > PATTERN.v

```
1  `ifdef RTL
2      `define CYCLE_TIME 20.0
3  `endif
4  `ifdef GATE
5      `define CYCLE_TIME 20.0
6  `endif
```

Fig. 9. 00_TESTBED/PATTERN.v

02_SYN > syn.tcl

```
1   #======================================================
2   #
3   # Synopsys Synthesis Scripts (Design Vision dctcl mode)
4   #
5   #======================================================
6
7   #======================================================
8   # (A) Global Parameters
9   #======================================================
10  set DESIGN "MATRIX_MULT"
11  set CYCLE 20
12  set INPUT_DLY [expr 0.5*$CYCLE]
13  set OUTPUT_DLY [expr 0.5*$CYCLE]
```

Fig. 10. 02_SYN/syn.tcl